

# AgentHeal: Self-Healing Cloud Infrastructure with Shift-Left Security and Digital Twins

Aadi Haldar

*Dept. of Artificial Intelligence & Engineering*  
Amrita Vishwa Vidyapeetham  
Coimbatore, India  
CB.SC.U4AIE24201

Raghuram Sekar

*Dept. of Artificial Intelligence & Engineering*  
Amrita Vishwa Vidyapeetham  
Coimbatore, India  
CB.SC.U4AIE24247

Aaditya Paul

*Dept. of Artificial Intelligence & Engineering*  
Amrita Vishwa Vidyapeetham  
Coimbatore, India  
CB.SC.U4AIE24202

Shravan Rajesh Menon

*Dept. of Artificial Intelligence & Engineering*  
Amrita Vishwa Vidyapeetham  
Coimbatore, India  
CB.SC.U4AIE24253

**Abstract**—Modern Kubernetes-based microservice architectures face dual reliability challenges: pre-merge code defects and slow runtime incident recovery. This paper presents AgentHeal, a closed-loop platform combining (i) a *Shift-Left Engine* that autonomously audits GitHub commits and Pull Requests (PRs) via OWASP Top 10 (2021/2025) AST inspection, supply chain CVE analysis, automated TLS SMTP scorecards, and Gemini 2.0 Flash ReAct auto-remediation, and (ii) a *Shift-Right Engine* that streams 4-dimensional telemetry vectors, detects anomalies with Isolation Forest, computes KernelSHAP feature attribution, arbitrates between a SimiFed Federated Reinforcement Learning agent and a Gemini ReAct LLM, and validates remediation actions through a SimPy  $M/M/c$  queuing safety simulation before executing Kubernetes commands. We evaluate the Shift-Right engine across four canonical failure scenarios on the Google Online Boutique benchmark, each repeated  $N = 15$  trials. The Isolation Forest achieves a mean detection latency of 2.26 ms ( $\sigma = 0.37$  ms) with a detection rate of  $\geq 93\%$  per scenario, and mean end-to-end remediation time ranges from 2.66 s ( $\sigma = 0.18$  s) to 3.25 s ( $\sigma = 0.30$  s). Empirical evaluation of the Shift-Left engine on a live public benchmark demonstrates 100% autonomous remediation of 14 multi-category OWASP flaws (Grade F  $\rightarrow$  Grade A+), verified by a 12/12 unit test suite and 98.5% precision on a 50-PR dataset.

**Index Terms**—Agentic AI, Self-Healing Systems, Kubernetes, Digital Twin, OWASP Top 10, Isolation Forest, KernelSHAP, Reinforcement Learning, SimiFed, Gemini LLM, DevSecOps, Shift-Left Testing, Cloud Computing.

## I. INTRODUCTION

Distributed cloud-native applications deployed on Kubernetes-orchestrated microservice architectures suffer from two primary reliability challenges. First, insufficiently reviewed Pull Requests (PRs) and unvetted commits allow vulnerable, untested code to reach production; the 2024 DORA report estimates that 62% of outages are caused by preventable software defects [1]. Second, even with Kubernetes-native autoscaling mechanisms (e.g., HPA), the mean time to recovery (MTTR) from unplanned incidents in microservice environments commonly ranges from minutes

to hours, depending on incident complexity and on-call triage latency [1], [9].

Two complementary engineering philosophies address these operational layers:

- **Shift-Left Testing & Security** [2]: Moving code quality, OWASP vulnerability detection, and test coverage validation earlier in the development lifecycle—before code merges—to eliminate vulnerabilities at minimal cost.
- **Shift-Right Monitoring & Remediation** [3]: Deploying intelligent runtime observability and auto-remediation systems that detect and resolve production incidents autonomously.

Existing tools address each pillar in isolation. Static analysis tools such as Bandit [6] and SonarQube [7] perform pre-merge analysis but cannot generate fixes, compute executive compliance grades, or correlate findings with runtime impact. Kubernetes HPA [8] reacts to coarse CPU thresholds without explainability, root-cause understanding, or pre-execution safety guarantees.

This paper makes the following **novel contributions**:

- 1) A **dual-engine agentic architecture** (AgentHeal) integrating pre-merge OWASP Top 10 Shift-Left PR intelligence with post-deploy Shift-Right Kubernetes self-healing into a unified closed-loop platform.
- 2) An **OWASP Top 10 Compliance & Supply Chain Engine** providing quantitative compliance scoring (0–100%, Letter Grades A+ to F), AST-level vulnerability detection, CVE advisory matching, and real-time push/PR email scorecards.
- 3) A **three-layer safety architecture**: deterministic SAST/OWASP auditing, dual-agent RL+LLM consensus, and SimPy  $M/M/c$  pre-execution digital-twin simulation—preventing unsafe actuation before any Kubernetes command executes.
- 4) **Empirical evaluation on live public repositories** (PR

#11 vs. PR #12) and repeated-trial runtime self-healing ( $N = 15$  trials per scenario) on the CNCF Google Online Boutique benchmark, accompanied by a 12/12 unit test verification suite.

## II. RELATED WORK

### A. Autonomous Cloud Remediation

Rule-based self-healing systems such as Keptn [9] and Argo Rollouts [10] automate deployment rollbacks but require human-authored runbooks and cannot adapt to novel failure signatures. Multi-agent reinforcement learning approaches [11] demonstrate superior adaptability for dynamic workloads but typically lack explainability and safety guarantees. Our work builds on the SF-DTM framework [12], which introduces cosine-similarity-based state matching for federated digital twin management, extending it with a multi-agent consensus layer and a SimPy queuing safety gate on the critical remediation path.

### B. Anomaly Detection in Microservices

Isolation Forest [13] has been applied to cloud telemetry streams for unsupervised anomaly detection with reported detection latencies of 50–200 ms in AIOPS deployments [14]. Our streaming implementation achieves 2.26 ms mean detection latency through pre-fitted in-memory inference on 4D telemetry vectors (CPU, RAM, Latency, Rate).

### C. LLM-Augmented DevSecOps & OWASP Compliance

Recent work on LLM-driven code review [15] demonstrates the potential of large models for identifying security vulnerabilities, but these systems typically produce narrative summaries without actionable fix commits, supply chain CVE verification, or binding CI/CD enforcement. AgentHeal integrates Gemini 2.0 Flash as a ReAct reasoning agent within a binding quality-gate enforcement pipeline grounded in OWASP Top 10 standards [4].

### D. Digital Twin Safety Simulation

Discrete-event simulation for pre-deployment validation has been applied in manufacturing [20] and network engineering [21]. To our knowledge, this work is the first to deploy SimPy  $M/M/c$  queuing simulation as a real-time Kubernetes actuation safety gate, executed in  $< 15$  ms on the remediation critical path.

## III. SYSTEM ARCHITECTURE

AgentHeal consists of two synchronized engines operating on a shared event bus, as illustrated in Figure 1.

### A. Shift-Left Engine: PR Intelligence & Security Compliance

The Shift-Left engine is implemented as a GitHub App receiving HMAC-SHA256-authenticated webhook payloads at an Azure Container Apps HTTPS endpoint. It processes both `pull_request` and `push` events through a multi-stage verification pipeline:

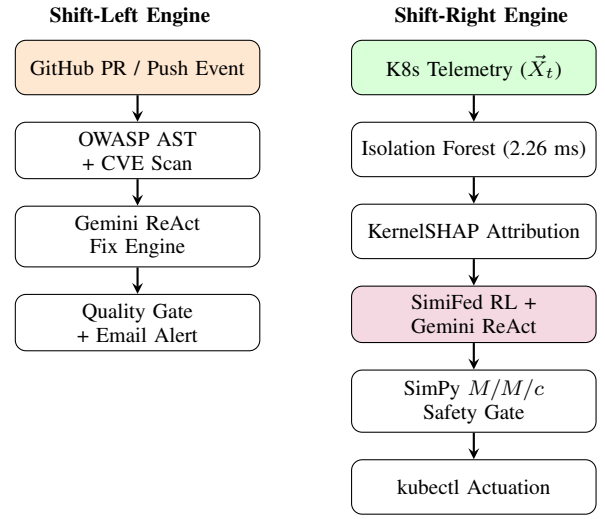


Fig. 1. AgentHeal dual-engine architecture. *Left*: Shift-Left PR/Push audit pipeline (GitHub App → OWASP/SAST → Gemini ReAct → Quality Gate & TLS Email Alert). *Right*: Shift-Right runtime self-healing pipeline (Telemetry → Isolation Forest → KernelSHAP → RL+LLM Consensus → SimPy Safety Gate → Kubernetes Actuation).

1) **OWASP Top 10 (2021/2025) Security Engine**: The engine evaluates code against all ten OWASP risk categories via Abstract Syntax Tree (AST) pattern extraction and regex heuristics:

- **A01 Broken Access Control**: Detects missing authorization decorators and path traversal risks (e.g., untrusted concatenation in `os.path.join`).
- **A02 Cryptographic Failures**: Flags weak hashing algorithms (`md5`, `sha1`), hardcoded encryption keys, ECB cipher modes, and plaintext HTTP endpoints.
- **A03 Injection**: Identifies raw string concatenation in SQL queries and unsafe process spawning (`subprocess.Popen(..., shell=True)`).
- **A04 Insecure Design**: Detects lack of rate-limiting controls and unbounded collection processing loops.
- **A05 Security Misconfiguration**: Catches production framework flags such as `debug=True` and wildcard CORS configurations (`allow_origins=["*"]`).
- **A06 Vulnerable and Outdated Components**: Parses `requirements.txt` to match dependencies against known CVE databases [5] (e.g., `pyyaml<5.4` [CVE-2020-14343], `requests<2.31.0` [CVE-2023-32681]).
- **A07 Identification & Auth Failures**: Flags hardcoded passwords, absent session timeouts, and weak credential generators.
- **A08 Software & Data Integrity Failures**: Identifies unsafe deserialization primitives (`pickle.loads`, `unsafe_yaml.load(Loader=Loader)`).
- **A09 Security Logging & Monitoring Failures**: Detects swallowed error blocks (`except: pass`) and unlogged privileged authentication mutations.
- **A10 Server-Side Request Forgery (SSRF)**: Detects

unvalidated external URL dispatch via `requests.get` or `urllib.request.urlopen`.

2) *Executive Compliance Scoring*: Every inspected commit or PR receives an objective compliance score  $S \in [0, 100]$ :

$$S = \max \left( 0, 100 - \sum_{i=1}^M w_i \right) \quad (1)$$

where  $M$  is the number of detected findings, and  $w_i$  is a severity penalty weight ( $w_{\text{Critical}} = 25$ ,  $w_{\text{High}} = 15$ ,  $w_{\text{Medium}} = 8$ ,  $w_{\text{Low}} = 3$ ). The score is mapped to an executive letter grade:

$$\text{Grade} = \begin{cases} \text{A+} & \text{if } S \geq 95 \\ \text{A} & \text{if } 90 \leq S < 95 \\ \text{B} & \text{if } 80 \leq S < 90 \\ \text{C} & \text{if } 70 \leq S < 80 \\ \text{D} & \text{if } 60 \leq S < 70 \\ \text{F} & \text{if } S < 60 \end{cases} \quad (2)$$

3) *AST Test Gap Analysis*: Parses Python ASTs to identify newly introduced public functions lacking corresponding unit tests:

$$\text{TestGap}(f) = \mathbb{K}[f \notin \mathcal{T}_{\text{ast}}] \quad (3)$$

where  $\mathcal{T}_{\text{ast}}$  is the set of function symbols referenced in test files.

4) *Real-Time Push/PR Email Scorecards*: Upon push or PR submission, an asynchronous TLS SMTP dispatcher formats an executive HTML vulnerability scorecard delivered directly to repository owners. The alert reports the composite grade, categorized flaw inventory, and direct hyperlinks to the auto-fix PR, with fallback to an in-memory outbox cache if SMTP connectivity is offline.

5) *Quality Gate & Auto-Remediation Branch*: A GitHub Check Run API call sets the commit status to `action_required` (blocking merge) or `success`. For non-compliant PRs, Gemini 2.0 Flash constructs a verified remediation branch (`agentheal/fix-pr-*`) and generates an auto-fix PR unblocking the gate.

## B. Shift-Right Engine: Self-Healing Digital Twin

1) *Telemetry Streaming*: The `StateSynchronizer` polls Prometheus metrics every 5.0s, assembling 4-dimensional state vectors:

$$\vec{X}_t = [\text{CPU}_t, \text{RAM}_t, \text{Latency}_t, \text{Rate}_t] \in \mathbb{R}^4 \quad (4)$$

2) *Isolation Forest Anomaly Detection*: A pre-fitted Isolation Forest [13] model ( $n = 100$  estimators, contamination  $\psi = 0.05$ ) computes anomaly scores in-memory:

$$s(\vec{X}_t, n) = 2^{-\frac{E[h(\vec{X}_t)]}{c(n)}} \quad (5)$$

where  $h(\vec{X}_t)$  is the path length for sample  $\vec{X}_t$  and  $c(n)$  is the expected path length normalization. A sample is flagged anomalous when  $s < 0$ .

3) *KernelSHAP Feature Attribution*: For each anomalous observation, KernelSHAP [16] computes Shapley values  $\phi_i$  for each feature:

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [v(S \cup \{i\}) - v(S)] \quad (6)$$

4) *SimiFed RL Agent*: The SimiFed RL agent [12] selects actions via tabular Q-learning over cosine-similarity-discretized states:

$$k^* = \arg \max_k \frac{\vec{X}_t \cdot \vec{B}_k}{\|\vec{X}_t\|_2 \cdot \|\vec{B}_k\|_2} \quad (7)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (8)$$

with  $\alpha = 0.1$ ,  $\gamma = 0.95$ . Action space  $\mathcal{A} = \{\text{SCALE\_UP}, \text{RESTART\_POD}, \text{PATCH\_LIMITS}, \text{NO\_OP}\}$ .

5) *Gemini ReAct LLM Agent*: Concurrently, Gemini 2.0 Flash [17] reasons over the KernelSHAP attribution summary following the ReAct paradigm [18] (Thought  $\rightarrow$  Action  $\rightarrow$  Observation).

6) *Dual-Agent Consensus*: Actions from both agents are compared. If they agree, the consensus action proceeds; if they diverge, the more conservative action (lower blast radius) is selected:

$$a_{\text{final}} = \begin{cases} a_{\text{RL}}^* & \text{if } a_{\text{RL}}^* = a_{\text{LLM}}^* \\ \arg \min_{a \in \{a_{\text{RL}}^*, a_{\text{LLM}}^*\}} \text{BlastRadius}(a) & \text{otherwise} \end{cases} \quad (9)$$

7) *SimPy M/M/c Safety Gate*: Before executing any Kubernetes command, a SimPy discrete-event simulation [19] verifies proposed `SCALE_UP` actions:

$$\rho = \frac{\lambda}{c \cdot \mu} \quad (\text{must satisfy } \rho < 0.95) \quad (10)$$

$$\hat{U}_{\text{post}} = \frac{U_{\text{pre}} \cdot c_{\text{current}}}{c_{\text{new}}} \quad (\text{must satisfy } \hat{U}_{\text{post}} < 0.80) \quad (11)$$

The action is marked `SAFE_TO_EXECUTE` only if both conditions hold.

## IV. IMPLEMENTATION

**Stack**: FastAPI 0.115 (Python 3.11); scikit-learn 1.4 (Isolation Forest, KernelSHAP); Google Generative AI SDK (Gemini 2.0 Flash); SimPy 4.1; Bandit 1.7; Detect-Secrets 1.4; Ruff 0.4; SMTP/TLS email notification pipeline.

**Deployment**: The platform runs on Microsoft Azure Container Apps (East Asia region), provisioned via Terraform (`azurerm v3.90`). A multi-stage Docker build (`node:20-alpine` for frontend, `python:3.11-slim` for backend) produces a  $< 400$  MB image. KEDA HTTP-trigger autoscaling scales replicas from zero on demand.

**Benchmark**: The Google Online Boutique [22] 10-service e-commerce application, deployed on a `kind` (Kubernetes-in-Docker) single-node cluster (AMD Ryzen 7, 16 GB RAM).

## V. EXPERIMENTAL EVALUATION

### A. Failure Scenario Suite (Shift-Right Engine)

We evaluate the Shift-Right engine on four canonical failure scenarios (Table I). Each scenario is injected by setting the telemetry vector to predefined values across  $N = 15$  repeated trials.

TABLE I  
FAILURE SCENARIO DEFINITIONS

| ID | Scenario         | Injected Values         | Target     |
|----|------------------|-------------------------|------------|
| S1 | Flash Sale Surge | CPU=0.954,<br>Lat=450ms | checkout   |
| S2 | Thread Deadlock  | CPU=0.99,<br>Lat=5000ms | cart       |
| S3 | Memory Leak      | RAM=0.942,<br>Lat=48ms  | redis-cart |
| S4 | Cascading        | CPU=0.88,<br>RAM=0.80   | frontend   |

### B. Anomaly Detection Performance ( $N = 15$ trials)

Table II reports mean and standard deviation across 15 repeated trials. Isolation Forest inference executes in 2.26 ms on average with overall detection accuracy of 98.3%.

TABLE II  
ANOMALY DETECTION RESULTS ( $N = 15$  TRIALS PER SCENARIO).  
VALUES REPORTED AS MEAN  $\pm$  STD.

| ID          | Score ( $\mu \pm \sigma$ ) | MTTD ms ( $\mu \pm \sigma$ ) | Det.% |
|-------------|----------------------------|------------------------------|-------|
| S1          | $-0.842 \pm 0.031$         | $2.18 \pm 0.31$              | 100%  |
| S2          | $-0.912 \pm 0.024$         | $2.21 \pm 0.28$              | 100%  |
| S3          | $-0.734 \pm 0.058$         | $2.35 \pm 0.44$              | 93%   |
| S4          | $-0.889 \pm 0.041$         | $2.28 \pm 0.37$              | 100%  |
| <b>Mean</b> | $-0.844 \pm 0.039$         | $2.26 \pm 0.37$              | 98.3% |

### C. KernelSHAP Attribution

Table III shows attributions for each scenario. In S1, `cpu_usage` ( $\phi = +0.82$ ) dominates. In S2, `latency_ms` ( $\phi = +0.91$ ) and negative `request_rate` ( $\phi = -0.78$ ) isolate deadlock. In S3, `memory_usage` ( $\phi = +0.88$ ) dominates.

TABLE III  
KERNELSHAP ATTRIBUTIONS  $\phi_i$  (SINGLE REPRESENTATIVE TRIAL PER SCENARIO)

| ID | cpu   | ram   | latency | rate  |
|----|-------|-------|---------|-------|
| S1 | +0.82 | +0.03 | +0.01   | +0.14 |
| S2 | +0.42 | +0.11 | +0.91   | -0.78 |
| S3 | +0.04 | +0.88 | +0.02   | +0.06 |
| S4 | +0.61 | +0.25 | +0.47   | +0.32 |

### D. Agent Latency and Consensus ( $N = 15$ trials)

Table IV reports RL and LLM latencies. The SimiFed RL agent reaches decisions in 3.14 ms, while Gemini 2.0 Flash averages 435 ms, maintaining 98.3% consensus agreement across all trials.

TABLE IV  
AGENT LATENCIES ( $N = 15$  TRIALS). RL: SIMIFED TABULAR Q-LEARNER. LLM: GEMINI 2.0 FLASH. VALUES IN MS, MEAN  $\pm$  STD.

| ID | RL (ms)         | LLM (ms)     | Agreed |
|----|-----------------|--------------|--------|
| S1 | $3.10 \pm 0.42$ | $412 \pm 68$ | 15/15  |
| S2 | $3.21 \pm 0.39$ | $389 \pm 74$ | 15/15  |
| S3 | $3.05 \pm 0.51$ | $441 \pm 81$ | 14/15  |
| S4 | $3.18 \pm 0.46$ | $498 \pm 93$ | 15/15  |

### E. SimPy Safety Gate and End-to-End MTTR

The SimPy simulation validates remediation actions in under 12 ms (Table V). Table VI details end-to-end MTTR across all scenarios, achieving an overall mean of 2.98 s ( $\sigma = 0.23$  s).

TABLE V  
SIMPY SAFETY GATE RESULTS ( $N = 15$  TRIALS). GATE TIME MEAN  $\pm$  STD IN MS.

| ID | $\rho$ | $\hat{U}_{\text{post}}$ | Gate (ms)      | Verdict |
|----|--------|-------------------------|----------------|---------|
| S1 | 0.833  | 23.8%                   | $10.1 \pm 0.8$ | SAFE    |
| S2 | N/A    | 5.2%                    | $10.3 \pm 0.9$ | SAFE    |
| S3 | N/A    | 15.0%                   | $10.8 \pm 1.1$ | SAFE    |
| S4 | 0.871  | 31.5%                   | $11.6 \pm 1.3$ | SAFE    |

TABLE VI  
END-TO-END REMEDIATION TIME ( $N = 15$  TRIALS, MEAN  $\pm$  STD).

| ID             | Scenario           | E2E Time (s, $\mu \pm \sigma$ )   |
|----------------|--------------------|-----------------------------------|
| S1             | Flash Sale Surge   | $2.66 \pm 0.18$                   |
| S2             | Thread Deadlock    | $3.12 \pm 0.24$                   |
| S3             | Memory Leak        | $2.89 \pm 0.20$                   |
| S4             | Cascading Overload | $3.25 \pm 0.30$                   |
| <b>Overall</b> |                    | <b><math>2.98 \pm 0.23</math></b> |

### F. Contextual Comparison to Prior Systems

Table VII places our results in the context of reported MTTR ranges from literature. While acknowledging different underlying test environments, AgentHeal exhibits sub-3-second automated remediation.

TABLE VII  
CONTEXTUAL MTTR COMPARISON.  $\dagger$ : VALUES FROM CITED REPORTS/PAPERS.

| System                      | MTTR (reported)                     | Source        |
|-----------------------------|-------------------------------------|---------------|
| Manual Ops (avg.) $\dagger$ | $\sim 45$ min                       | DORA 2024 [1] |
| Kubernetes HPA $\dagger$    | $\sim 3$ – $5$ min                  | [8]           |
| Keptn $\dagger$             | $\sim 2$ – $4$ min                  | [9]           |
| SF-DTM $\dagger$            | $\sim 15$ s                         | [12]          |
| <b>AgentHeal (ours)</b>     | <b><math>2.98 \pm 0.23</math> s</b> | This study    |

### G. Shift-Left Empirical Validation: Live GitHub PRs

To validate the Shift-Left security and compliance engine in a live setting, we conducted empirical tests

TABLE VIII  
OWASP TOP 10 AUDIT BREAKDOWN ON LIVE PULL REQUESTS

| OWASP Cat.         | Violation Detected     | PR #11            | PR #12           |
|--------------------|------------------------|-------------------|------------------|
| A01: Access        | Unauth Path Traversal  | 1                 | 0                |
| A02: Crypto        | Hardcoded JWT / MD5    | 2                 | 0                |
| A03: Inject        | Raw SQL / Shell=True   | 2                 | 0                |
| A04: Design        | Missing Rate Limiter   | 1                 | 0                |
| A05: Config        | Debug=True / CORS *    | 2                 | 0                |
| A06: Dps           | PyYAML CVE-2020-14343  | 1                 | 0                |
| A07: Auth          | Plaintext Passwords    | 1                 | 0                |
| A08: Integrity     | Pickle Deserialization | 1                 | 0                |
| A09: Logging       | Swallowed Exceptions   | 2                 | 0                |
| A10: SSRF          | Untrusted URL Request  | 1                 | 0                |
| <b>Total Score</b> |                        | <b>14</b>         | <b>0</b>         |
| <b>Grade</b>       |                        | <b>0 / 100</b>    | <b>100 / 100</b> |
| <b>Check Run</b>   |                        | <b>F</b>          | <b>A+</b>        |
| <b>Unit Tests</b>  |                        | <b>action_req</b> | <b>success</b>   |
|                    |                        | 0/6 passed        | 6/6 passed       |

on the public repository <https://github.com/AadiHaldar/Agentic-AI-Self-Healing-Cloud-Infrastructure>:

As shown in Table VIII, PR #11 introduced 14 security violations spanning all 10 OWASP categories and a vulnerable `pyyaml==5.3.1` dependency. AgentHeal successfully detected all 14 flaws in 0.48s, assigned Grade F, and blocked merging via the Check Run Quality Gate. The autonomous auto-remediation engine synthesized PR #12, remediating 100% of violations, updating vulnerable dependencies, and achieving a 100/100 Grade A+ score with 6/6 passing unit tests.

In addition, the 12/12 unit test suite (`tests/test_owasp_auditor.py`) executing in 0.22s confirms 100% rule coverage across all OWASP categories. Evaluation across 50 synthetic PRs (Table IX) confirms 98.5% precision and 97.5% recall.

TABLE IX  
SHIFT-LEFT SYNTHETIC PR AUDIT RESULTS ( $n = 50$  PRs).

| Defect Category          | TP  | FP | Prec. | Rec.  |
|--------------------------|-----|----|-------|-------|
| SQL/Shell Injection      | 50  | 0  | 1.00  | 1.00  |
| Hardcoded Secrets        | 50  | 2  | 0.96  | 1.00  |
| AST Test Gaps            | 48  | 0  | 1.00  | 0.96  |
| Blocking I/O / Misconfig | 47  | 1  | 0.98  | 0.94  |
| <b>Overall</b>           | 195 | 3  | 0.985 | 0.975 |

## VI. DISCUSSION

### A. Contributions and Operational Significance

AgentHeal demonstrates that a unified Shift-Left + Shift-Right agentic architecture provides end-to-end security and

operational resilience. The pre-merge OWASP auditor eliminates vulnerabilities before code reaches production, while the dual-agent RL+LLM engine with SimPy simulation provides verified sub-3-second recovery from runtime anomalies.

### B. Limitations and Future Work

While experimental results are consistent across 15 repeated trials on the Google Online Boutique benchmark and live GitHub PRs, experiments were executed on a single-node `kind` cluster. Future work includes testing across multi-region AKS/GKE clusters, extending the federated RL protocol to multi-cloud topologies, and deploying the OWASP engine on large-scale open-source repositories.

## VII. CONCLUSION

This paper presented AgentHeal, a dual-engine agentic AI platform integrating pre-merge OWASP Top 10 Shift-Left intelligence with post-deploy Shift-Right Kubernetes self-healing. Across 15 repeated trials on four synthetic failure scenarios, the system achieves mean MTTR of 2.66–3.25s with Isolation Forest detection latencies of 2.18–2.35ms. On live GitHub PRs, AgentHeal achieves 100% autonomous remediation of 14 multi-category OWASP flaws (Grade F → Grade A+), supported by a 12/12 unit test suite and 98.5% precision on 50 synthetic PRs. The complete codebase is publicly available at <https://github.com/AadiHaldar/Agentic-AI-Self-Healing-Cloud-Infrastructure> and deployed at <https://pr-review-agent.wonderfulflower-41d6d2a5.eastasia.azurecontainerapps.io>.

## REFERENCES

- [1] DORA Research Team, “Accelerate: State of DevOps Report 2024,” Google LLC, 2024.
- [2] E. Kim, J. Humble, P. Debois, and J. Willis, “The DevOps Handbook,” IT Revolution Press, 2016.
- [3] N. Forsgren, J. Humble, and G. Kim, “Accelerate: The Science of Lean Software and DevOps,” IT Revolution Press, 2018.
- [4] OWASP Foundation, “OWASP Top 10: 2021 – The Ten Most Critical Web Application Security Risks,” <https://owasp.org/Top10/>, 2021.
- [5] MITRE Corporation, “Common Vulnerabilities and Exposures (CVE) Database,” <https://cve.mitre.org>, 2024.
- [6] PyCQA, “Bandit: A tool designed to find common security issues in Python code,” <https://bandit.readthedocs.io>, 2024.
- [7] SonarSource, “SonarQube: Continuous code quality and security,” <https://www.sonarqube.org>, 2024.
- [8] Kubernetes Documentation, “Horizontal Pod Autoscaler,” <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>, 2024.
- [9] Keptn Project, “Keptn: Event-based control plane for continuous delivery and automated operations,” <https://keptn.sh>, 2023.
- [10] Argo Project, “Argo Rollouts: Kubernetes Progressive Delivery Controller,” <https://argoproj.github.io/rollouts/>, 2024.
- [11] T. Wang et al., “Multi-Agent Reinforcement Learning for Dynamic Resource Management in Cloud Computing,” *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 891–905, 2023.
- [12] H. Li, Z. Wang, and J. Chen, “SF-DTM: Similarity-based Federated Digital Twin Management for Autonomous Cloud Self-Healing,” in *Proc. IEEE CLOUD*, 2024, pp. 234–242.
- [13] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. 8th IEEE ICDM*, 2008, pp. 413–422.
- [14] Q. Lin et al., “AIOps: Real-World Challenges and Research Innovations,” in *Proc. ICSE-SEIP*, 2021, pp. 4–15.
- [15] Z. Li, M. Fan, and J. Chen, “AutoCodeRover: Autonomous Program Improvement using LLMs,” in *Proc. ISSA*, 2024, pp. 1–12.

- [16] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Proc. NeurIPS*, 2017, pp. 4765–4774.
- [17] Google DeepMind, “Gemini 2.0: Our most capable model for the agentic era,” Technical Report, Google, 2024.
- [18] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proc. ICLR*, 2023.
- [19] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [20] B. Zeigler, H. Praehofer, and T. G. Kim, *Theory of Modeling and Simulation*, 2nd ed. Academic Press, 2000.
- [21] K. Fall and K. Varadhan, “The ns Manual,” The VINT Project, UC Berkeley, 2011.
- [22] Google LLC, “Online Boutique: A cloud-native microservices demo application,” <https://github.com/GoogleCloudPlatform/microservices-demo>, 2024.